

Bản sơ lược về các bài toán thống kê theo chữ số

Lưu Thông
Trường Trung học số 2 Thanh Đảo, Sơn Đông

2009

Nguồn gốc: On digit-statistics problems

I0I-China-Candidate-Team-Papers/2009/Liu-Cong/digit-statistics.pdf

Abstract

Trong thi tin học có một lớp bài toán thống kê trên đoạn liên quan đến chữ số. Những bài này thường mang màu sắc toán học khá rõ, không thể giải bằng vét cạn mà cần thực hiện truy hồi và các thao tác tương tự trên từng chữ số. Qua một vài ví dụ, bài viết giới thiệu ngắn gọn các tư tưởng và phương pháp cơ bản để giải lớp bài toán này.

Từ khóa: chữ số, đoạn, thống kê, truy hồi, cây, nhị phân.

1 Mở đầu

Trong thi tin học có một lớp bài toán như sau: cho một đoạn, yêu cầu tìm một số ở hệ cơ số D thỏa mãn điều kiện cho trước, hoặc đếm số lượng các số như vậy. Điều kiện cần xét thường liên quan đến chữ số, chẳng hạn tổng chữ số, số lượng một chữ số chỉ định, cách chia nhóm theo thứ tự lớn nhỏ của số, v.v. Đoạn được cho trong đề thường rất lớn, nên không thể dùng phương pháp thô sơ.

Khi đó ta cần lợi dụng tính chất của chữ số để thiết kế thuật toán có độ phức tạp cỡ $\log n$. Tư tưởng cơ bản nhất để giải loại bài toán này là phương pháp **xác định từng chữ số**. Dưới đây ta xét một số bài mẫu để hiểu cụ thể hơn về dạng bài và cách suy nghĩ này.

2 Ví dụ 1: Amount of degrees (URAL 1057)

2.1 Đề bài

Cho đoạn $[X, Y]$. Hãy đếm số nguyên trong đoạn này thỏa mãn điều kiện: số đó bằng tổng của đúng K lũy thừa nguyên khác nhau của B . Chẳng hạn, với $X = 15$, $Y = 20$, $K = 2$, $B = 2$, chỉ có ba số sau thỏa mãn:

$$17 = 2^4 + 2^0, \quad 18 = 2^4 + 2^1, \quad 20 = 2^4 + 2^2.$$

Dữ liệu vào gồm hai số nguyên X, Y ở dòng đầu, sau đó là K và B . Dữ liệu ra là một số nguyên, tức số lượng số thỏa mãn. Ràng buộc:

$$1 \leq X \leq Y \leq 2^{31} - 1, \quad 1 \leq K \leq 20, \quad 2 \leq B \leq 10.$$

2.2 Phân tích

Số cần tìm là tổng của các lũy thừa khác nhau, nghĩa là trong biểu diễn cơ số B của nó, mọi chữ số chỉ có thể là 0 hoặc 1. Vì vậy, trước hết ta chỉ cần thảo luận trường hợp nhị phân; các cơ số khác đều có thể chuyển về một bài toán nhị phân tương ứng.

Rõ ràng miền dữ liệu rất lớn, không thể liệt kê. Độ phức tạp phải ở cấp $\log n$, nên ta phải xử lý trên chữ số.

Bài này thỏa mãn tính chất trừ theo đoạn. Gọi $\text{count}[i..j]$ là số lượng số hợp lệ trong đoạn $[i, j]$, khi đó

$$\text{count}[i..j] = \text{count}[0..j] - \text{count}[0..i - 1].$$

Nói cách khác, với một n cho trước, ta chỉ cần tính có bao nhiêu số từ 0 đến n thỏa mãn điều kiện.

Giả sử $n = 13$, biểu diễn nhị phân là 1101, và $K = 3$. Mục tiêu là đếm các số từ 0 đến 13 có biểu diễn nhị phân chứa đúng 3 bit 1. Để dễ suy nghĩ, hãy tưởng tượng một cây nhị phân đầy đủ cao 4, trong đó mỗi cạnh hoặc mỗi bước đi tương ứng với chọn chữ số 0 hoặc 1. Mỗi lá biểu diễn một số nhị phân 4 chữ số, tức các số từ 0 đến $2^4 - 1$.

Đường đi đến lá biểu diễn n chia những số nhỏ hơn n thành một số cây con đầy đủ. Mỗi khi trên đường đi ta rẽ phải, toàn bộ cây con bên trái tại vị trí đó đều gồm các số nhỏ hơn n . Do đó để đếm các số nhỏ hơn 13, ta chỉ cần đếm trong các cây con đầy đủ này: có cây cần đếm số có 3 bit 1, có cây chỉ cần thêm 2 bit 1 vì trên tiền tố đã có một bit 1, và có cây chỉ cần thêm 1 bit 1. Các cây con có cùng chiều cao thì kết quả thống kê như nhau. Đừng quên xét chính n ; trong cài đặt, có thể tăng n lên 1 ngay từ đầu để tránh bàn riêng trường hợp này, nhưng phải chú ý tràn số.

Còn lại là bài toán: trong một cây nhị phân đầy đủ cao i , có bao nhiêu lá mà biểu diễn nhị phân chứa đúng j bit 1? Điều này được tính dễ dàng bằng truy hồi. Đặt $f[i, j]$ là đáp án. Tách theo cây con trái và phải, ta có

$$f[i, j] = f[i - 1, j] + f[i - 1, j - 1].$$

Vậy thuật toán trả lời là: tiền xử lý f , rồi với n đầu vào, đi từ gốc đến lá tương ứng trong cây đầy đủ tưởng tượng; mỗi khi rẽ phải thì cộng số lượng số trong cây con trái. Mã C++ trong bài gốc như sau:

```
void init() // tien xu ly f
{
    f[0][0]=1;
    for (int i=1;i<=31;++i) {
        f[i][0]=f[i-1][0];
        for (int j=1;j<=i;++j) f[i][j]=f[i-1][j]+f[i-1][j-1];
    }
}

int calc(int x,int k) // dem trong [0..x] cac so co k bit 1
{
    int tot=0,ans=0; // tot: so bit 1 da co tren duong di
    for (int i=31;i>0;--i)
    {
        if (x&(1<<i)) {
            ++tot;
            if (tot>k) break;
            x=x^(1<<i);
        }
    }
}
```

```

    if ((1 < (i-1)) <= x) {
        ans += f[i-1][k-tot];
    }
}
if (tot+x==k) ++ans;
return ans;
}

```

Vấn đề cuối cùng là xử lý cơ số không phải nhị phân. Với truy vấn n , ta cần tìm số lớn nhất không vượt quá n mà biểu diễn cơ số B chỉ chứa 0 và 1: tìm chữ số đầu tiên từ trái sang phải không phải 0 hoặc 1, đổi nó thành 1, và đặt tất cả chữ số bên phải bằng 1. Sau đó xem biểu diễn cơ số B thu được như một biểu diễn nhị phân rồi trả lời bằng thuật toán trên.

Tiền xử lý truy hồi f có độ phức tạp $\mathcal{O}(\log^2 n)$. Có $\mathcal{O}(\log n)$ lần truy vấn, nên tổng độ phức tạp thời gian là $\mathcal{O}(\log^2 n)$.

Thực ra mã cuối cùng không thao tác trên cây; ta chỉ dùng hình ảnh cây để tiện suy nghĩ. Cũng có thể xét trực tiếp từ góc nhìn chữ số: với truy vấn n , tìm một vị trí có chữ số bằng 1 và đổi nó thành 0; các chữ số bên phải được chọn tùy ý, và ta cần đếm trong đó các cách có đúng $K - \text{tot}$ bit 1, trong đó tot là số bit 1 ở bên trái vị trí này. Khi đó có thể dùng công thức tổ hợp. Liệt kê từng vị trí bằng 1 để cộng dồn là đủ.

Ta thấy f suy ra ở trên chính là các số tổ hợp. Cả hai cách đều dùng tư tưởng xác định từng chữ số. Khi cảm thấy suy luận trực tiếp trên chữ số hơi khó, hãy vẽ hình để làm rõ ý tưởng.

3 Ví dụ 2: Sorted bit sequence (SPOJ 1182)

3.1 Đề bài

Sắp xếp tất cả số nguyên trong đoạn $[m, n]$ theo số lượng bit 1 trong biểu diễn nhị phân tăng dần. Nếu hai số có cùng số bit 1, sắp xếp theo giá trị của số. Hãy tìm số thứ k trong dãy sau khi sắp xếp. Số âm dùng biểu diễn bù hai: biểu diễn nhị phân của một số âm cộng với biểu diễn nhị phân của số đối của nó đúng bằng 2^{32} .

Ví dụ, với $m = 0, n = 5$, dãy sau khi sắp xếp là:

STT	số thập phân	biểu diễn nhị phân 32 bit
1	0	0000 0000 0000 0000 0000 0000 0000 0000
2	1	0000 0000 0000 0000 0000 0000 0000 0001
3	2	0000 0000 0000 0000 0000 0000 0000 0010
4	4	0000 0000 0000 0000 0000 0000 0000 0100
5	3	0000 0000 0000 0000 0000 0000 0000 0011
6	5	0000 0000 0000 0000 0000 0000 0000 0101

Với $m = -5, n = -2$, dãy là:

STT	số thập phân	biểu diễn nhị phân 32 bit
1	-4	1111 1111 1111 1111 1111 1111 1111 1100
2	-5	1111 1111 1111 1111 1111 1111 1111 1011
3	-3	1111 1111 1111 1111 1111 1111 1111 1101
4	-2	1111 1111 1111 1111 1111 1111 1111 1110

Dữ liệu gồm nhiều bộ test. Dòng đầu là số bộ test, không quá 1000. Mỗi bộ test chứa ba số nguyên

m, n, k . Với mỗi bộ test, in ra số thứ k trong dãy đã sắp xếp. Ràng buộc:

$$mn \geq 0, \quad -2^{31} \leq m \leq n \leq 2^{31} - 1, \quad 1 \leq k \leq \min\{n - m + 1, 2147473547\}.$$

3.2 Phân tích

Trước hết xét trường hợp m, n đều dương.

Vì khóa sắp xếp thứ nhất là số bit 1, khóa thứ hai là giá trị của số, ta dễ xác định đáp án có bao nhiêu bit 1: lần lượt thống kê trong đoạn $[m, n]$ số lượng số có 0, 1, 2, ... bit 1 cho đến khi tổng cộng dồn vượt quá k . Giá trị hiện tại chính là số bit 1 của đáp án; giả sử là s . Thuật toán ở ví dụ 1 giải được bước thống kê này. Đồng thời, ta cũng biết đáp án là số thứ mấy trong các số thuộc $[m, n]$ có đúng s bit 1.

Do đó chỉ cần nhị phân đáp án. Với một giá trị thử ans , tính số lượng số trong $[m, ans]$ có đúng s bit 1 để quyết định hướng tìm. Mỗi lần thống kê có độ phức tạp $\mathcal{O}(\log n)$, nên phần nhị phân có độ phức tạp $\mathcal{O}(\log^2 n)$; tiền xử lý cũng cùng cấp, vì thế thuật toán khá lý tưởng.

Trường hợp $m < 0$ cũng không khó xử lý. Ta có thể bỏ qua bit cao nhất của mọi số, tìm đáp án rồi đặt bit cao nhất trở lại bằng 1. Hoặc cũng có thể trực tiếp xem số âm là số nguyên không dấu 32 bit và xử lý giống số dương. Cả hai cách đều cần xử lý riêng trường hợp $n = 0$.

4 Ví dụ 3: Sequence (SPOJ 2319)

4.1 Đề bài

Cho tất cả số nhị phân K bit:

$$0, 1, \dots, 2^K - 1.$$

Cần chia dãy này thành đúng M nhóm, mỗi nhóm là một đoạn liên tiếp của dãy ban đầu. Gọi S_i ($1 \leq i \leq M$) là tổng số bit 1 trong biểu diễn nhị phân của tất cả số thuộc nhóm thứ i , và gọi S là giá trị lớn nhất trong các S_i . Nhiệm vụ là làm cho S nhỏ nhất.

Dữ liệu vào gồm hai số nguyên K, M . Dữ liệu ra là giá trị nhỏ nhất của S ; đáp án không bảo đảm lưu được trong kiểu số nguyên 64 bit. Ràng buộc:

$$1 \leq K \leq 100, \quad 1 \leq M \leq 100, \quad M \leq 2^K.$$

4.2 Phân tích

Làm trực tiếp không dễ, nhưng có thể xử lý bằng cách nhị phân đáp án.

Hiển nhiên, nếu biết giá trị S , ta có thể tham lam tìm số nhóm ít nhất mà dãy cần được chia. Khi S tăng, số nhóm này không tăng. Dựa vào tính đơn điệu đó, ta nhị phân S , rồi tham lam tính số đoạn ít nhất cần dùng để phủ $[0, 2^K - 1]$ và kiểm tra.

Còn lại là cách tính số nhóm. Vì M nhỏ, mỗi lần ta bắt đầu từ điểm hiện tại a và tìm b lớn nhất sao cho

$$\text{count}(b) - \text{count}(a - 1) \leq S,$$

trong đó $\text{count}(i)$ là tổng số bit 1 trong biểu diễn nhị phân của mọi số từ 0 đến i . Sau đó lấy $b + 1$ làm điểm bắt đầu mới, cho đến khi một lần tìm được b vượt quá $2^K - 1$, hoặc số đoạn đã vượt quá M .

Bằng phương pháp xác định từng chữ số, ta dễ tính $\text{count}(i)$. Tương tự ví dụ 1, trong cây nhị phân đầy đủ cao K , đi từ gốc đến lá i ; mỗi khi rẽ phải thì cộng trọng số của cây con trái. Nói cách khác, tìm mọi vị trí bit 1 của i và cộng trọng số tương ứng; khi cộng phải chú ý số bit 1 đã xuất hiện ở tiền tố trước đó.

Cách tìm b cũng không khó. Đặt $\text{find}(i)$ là giá trị k lớn nhất sao cho tổng số bit 1 trong các số từ 0 đến k không vượt quá i . Khi đó

$$b = \text{find}(\text{count}(a - 1) + S).$$

Hàm $\text{find}(i)$ cũng dùng phương pháp xác định từng chữ số: đi từ gốc xuống lá trong cây; nếu trọng số đã có cộng với trọng số cây con trái của nút hiện tại không vượt quá i thì đi sang phải, ngược lại đi sang trái.

Cả $\text{count}(i)$ và $\text{find}(i)$ đều có độ phức tạp $\mathcal{O}(K)$, nên tổng độ phức tạp thuật toán là $\mathcal{O}(MK \log S)$, cộng thêm chi phí của số học độ chính xác cao.

5 Ví dụ 4: Tickets (SGU 390)

5.1 Đề bài

Một người bán vé bán vé cho hành khách. Với mỗi hành khách, anh ta bán nhiều vé liên tiếp, cho đến khi tổng chữ số của các số hiệu vé đã bán không nhỏ hơn một số dương k cho trước. Sau đó anh ta lặp lại quy tắc đó với hành khách tiếp theo. Ban đầu người bán vé có các vé mang số nguyên liên tiếp từ L đến R . Hãy tính anh ta có thể bán vé cho bao nhiêu hành khách.

Dữ liệu vào gồm ba số nguyên L, R, k . Dữ liệu ra là đáp án. Ràng buộc:

$$1 \leq L \leq R \leq 10^{18}, \quad 1 \leq k \leq 1000.$$

5.2 Phân tích

Bài này hơi giống ví dụ 3: ta cũng chia các số nguyên trong một đoạn thành các nhóm liên tiếp. Khác biệt là ở đây k nhỏ, tức số nhóm có thể rất lớn, nên không thể tính từng nhóm một.

Như trước, ta bắt đầu từ trường hợp đặc biệt. Giả sử đoạn đã cho là $[1, 10^h - 1]$, tức một cây thập phân đầy đủ cao h . Hãy xét cách ghép các bài toán con thành bài toán gốc. Khó khăn gặp phải là việc “nối” giữa hai cây con: nhóm cuối của cây con trước chưa chắc đã đầy, nên vài phần tử đầu của cây con sau có thể bị ghép vào nhóm cuối ấy. Để giải quyết, ta thêm một chiều để ghi trước mỗi cây con còn bao nhiêu “chỗ trống”. Từ đó có truy hồi sau.

Đặt $f[h, \text{sum}, \text{rem}]$ là kết quả đối với một cây thập phân đầy đủ cao h , trong đó mỗi số thuộc đoạn này cần được cộng thêm tổng chữ số sum ở tiền tố, và trước đoạn này nhóm hiện tại còn rem đơn vị trống. Giá trị trả về của f cần ghi hai số: $f[h, \text{sum}, \text{rem}].a$ là số nhóm tạo ra, còn $f[h, \text{sum}, \text{rem}].b$ là phần trống còn lại của nhóm cuối, để có thể ghép với cây con kế tiếp.

Giá trị f được suy ra từ mười cây con:

$$\begin{aligned} \text{for } i := 0 \text{ to } 9 \text{ do } \quad & f[h, \text{sum}, \text{rem}] := \text{merge}(f[h, \text{sum}, \text{rem}], \\ & f[h - 1, \text{sum} + i, f[h, \text{sum}, \text{rem}].b]). \end{aligned}$$

Hàm merge biểu thị ghép hai bản ghi, với giả mã:

```
function merge(x, y);
  x.a := x.a + y.a;
  x.b := y.b;
  return x;
```

Khi $h = 0$, nếu $\text{rem} > 0$ thì $f[h, \text{sum}, \text{rem}].a = 0$, ngược lại $f[h, \text{sum}, \text{rem}].a = 1$; giá trị b cần tính theo quan hệ giữa sum và k .

Như vậy ta đã tính được số nhóm của các cây con đầy đủ. Công việc tiếp theo dễ hơn: trong cây thập phân đầy đủ, đi từ lá L đến lá R , tính và ghép các cây con đầy đủ gặp trên đường. Cụ thể, đi từ lá chứa L lên đến tầng ngay dưới tổ tiên chung gần nhất của L và R , trên đường đó tính các cây anh em ở bên phải. Sau đó, tại tầng ngay dưới LCA, đi từ tổ tiên của L sang tổ tiên của R và tính các cây con đi qua. Cuối cùng đi xuống lá R , trên đường tính các cây anh em ở bên trái. Tất nhiên cũng không được quên tính chính các lá chứa L và R .

Trong hình minh họa của bài gốc, tác giả dùng cây nhị phân để vẽ, nhưng ý tưởng hoàn toàn giống cây thập phân: trước hết tìm LCA của L và R , đi ngược từ L lên và cộng các cây con bên phải, rồi đi sang phía tổ tiên của R , cuối cùng đi xuống R và cộng các cây con bên trái.

Cần xử lý riêng các trường hợp $L = R$ và $R = 10^{18}$ nếu chỉ tính đến 18 tầng. Ngoài ra, nếu nhóm cuối cùng chưa đầy thì không tính vào tổng số nhóm; điểm này cần đặc biệt chú ý.

Độ phức tạp của truy hồi tính f là $\mathcal{O}(k \log^2 R)$. Số cây con cần hỏi là $\mathcal{O}(\log R)$, nên tổng độ phức tạp do phần truy hồi chi phối, tức $\mathcal{O}(k \log^2 R)$. Nên cài đặt truy hồi bằng tìm kiếm có nhớ.

6 Ví dụ 5: Graduated Lexicographical Ordering (ZJU 2599)

6.1 Đề bài

Xét mọi số nguyên từ 1 đến n . Gọi $w(x)$ là tổng các chữ số thập phân của số nguyên x .

Ta sắp xếp lại các số như sau. Với hai số nguyên a và b , nếu $w(a) < w(b)$ thì a đứng trước b . Nếu $w(a) = w(b)$ thì a đứng trước b khi và chỉ khi biểu diễn thập phân của a nhỏ hơn biểu diễn thập phân của b theo thứ tự từ điển. Ví dụ:

$$120 <_{\text{grlex}} 4 \quad \text{vì} \quad w(120) = 1 + 2 + 0 = 3 < 4 = w(4),$$

$$555 <_{\text{grlex}} 78 \quad \text{vì} \quad w(555) = 15 = w(78) \text{ và "555" nhỏ hơn "78" theo từ điển,}$$

$$20 <_{\text{grlex}} 200 \quad \text{vì} \quad w(20) = 2 = w(200) \text{ và "20" nhỏ hơn "200" theo từ điển.}$$

Cho n và k , cần tìm thứ hạng của số k trong dãy các số từ 1 đến n sắp theo thứ tự trên, đồng thời tìm số đứng ở vị trí thứ k .

Dữ liệu gồm nhiều bộ, mỗi bộ có hai số nguyên n, k ; dòng cuối cùng là $n = k = 0$. Mỗi bộ in ra hai số nguyên: thứ hạng của k , và số đứng ở vị trí thứ k . Ràng buộc:

$$1 \leq k \leq n \leq 10^{18}.$$

6.2 Phân tích

Trước hết xét câu hỏi thứ nhất: cho một số k , tìm thứ hạng của nó. Mọi số có tổng chữ số nhỏ hơn $\text{sumof}(k)$ đều đứng trước k , nên ta cộng số lượng các số này. Vì vậy cần một hàm $\text{count}(n, i)$, đếm trong $[1, n]$ có bao nhiêu số có tổng chữ số bằng i . Cách cài đặt tương tự các ví dụ trước nên không nhắc lại chi tiết.

Sau đó, ta cần đếm trong $[1, n]$ các số có tổng chữ số bằng $\text{sumof}(k)$ và nhỏ hơn k theo thứ tự từ điển.

Với hai số có cùng số chữ số, thứ tự từ điển chính là thứ tự theo giá trị. Nhưng vì số không được có chữ số 0 ở đầu, so sánh các số khác độ dài sẽ rất phiền. Để tránh điều này, ta chỉ so sánh các số có cùng độ dài: lần lượt xét $k, k \cdot 10, k \cdot 100, \dots$ và $k/10, k/100, \dots$; tức là liệt kê mọi độ dài có thể của k trong phạm vi 1 đến n bằng cách thêm chữ số 0 hoặc bỏ chữ số ở cuối.

Với mỗi k' ở một độ dài cố định, số lượng số có cùng độ dài, nhỏ hơn k' và có tổng chữ số bằng $\text{sumof}(k)$ là

$$\text{count}(k', \text{sumof}(k)) - f[\text{lengthof}(k') - 1, \text{sumof}(k)].$$

Ở đây $\text{sumof}(k)$ là tổng chữ số của k , $\text{lengthof}(k')$ là độ dài thập phân của k' , và $f[i, j]$ là dữ liệu tiền xử lý khi tính $\text{count}(n, i)$: số lượng số trong đoạn $[0, 10^i - 1]$ có tổng chữ số bằng j . Như vậy ta giải được câu hỏi thứ nhất.

Tiếp theo xét câu hỏi thứ hai. Lần lượt trừ khỏi k các giá trị

$$\text{count}(n, 1), \text{count}(n, 2), \dots$$

cho đến khi k sắp nhỏ hơn 0; khi đó xác định được tổng chữ số s của đáp án. Việc còn lại là tìm số có tổng chữ số bằng s và có thứ tự từ điển thứ k trong $[1, n]$.

Đáng tiếc là vì thứ tự từ điển khác thứ tự số học, ta không thể dùng câu hỏi thứ nhất để nhị phân. Có thể dùng một phương pháp xác định từng chữ số khá trực tiếp: xác định đáp án từ trái sang phải. Vì chưa biết độ dài đáp án, ta trước hết xác định chữ số đầu tiên bằng cách liệt kê chữ số đầu và dùng thuật toán của câu hỏi thứ nhất để kiểm tra số lượng có vượt quá k hay không. Sau đó mỗi lần thêm một chữ số vào bên phải đáp án hiện có. Nếu sau khi thêm, số lượng số hợp lệ đúng bằng k thì trả về đáp án hiện tại; nếu không thì tiếp tục thêm cho đến khi tìm được nghiệm. Vì đáp án chắc chắn tồn tại, quá trình thêm chữ số không vượt quá $\mathcal{O}(\log n)$ lần.

7 Tổng kết

Qua các ví dụ trên có thể thấy, các thao tác cơ bản trong bài toán chữ số khá tương tự nhau và có tính mô hình rõ rệt. Trên nền các thao tác cơ bản này có thể biến hóa thành nhiều kiểu bài khác nhau.

Tư tưởng cốt lõi khi giải là xác định từng chữ số. Cách suy nghĩ thông thường là xét trực tiếp trên các chữ số. Tương ứng với nó là cách nhìn bằng cây, xem các số như những lá của một cây đầy đủ. Cách thứ nhất trừu tượng hơn, còn cách thứ hai cụ thể hơn. Vì vậy khi đề bài hơi rườm rà, dùng cách nhìn bằng cây thường có thể biến trừu tượng thành cụ thể và làm mạch suy nghĩ rõ ràng hơn.

Do các thao tác cơ bản có độ phức tạp cấp $\mathcal{O}(\log n)$, khi xử lý một số bài phức tạp hơn ta có thể hy sinh một phần thời gian, dùng nhị phân hoặc liệt kê cho một vài bài toán con để giảm độ phức tạp suy nghĩ và lập trình.

Ở đây tôi chọn vài ví dụ khá tiêu biểu để giới thiệu các bài toán thống kê theo chữ số, hy vọng có thể gợi ý cho người đọc. Cảm nhận và kỹ xảo sâu hơn vẫn cần được tích lũy dần trong quá trình làm bài.

Lời cảm ơn

Xin cảm ơn thầy Tạng Phương Thanh của Trường Trung học số 2 Thanh Đảo đã hướng dẫn tôi. Xin cảm ơn huấn luyện viên Đường Văn Bản đã cung cấp ví dụ 4, bạn Đồng Hoa Tinh của Trường Trung học số 1 Thiệu Hưng đã cung cấp ví dụ 2, và tất cả thầy cô, bạn học đã quan tâm, ủng hộ tôi.

A Đề gốc của ví dụ 1: Amount of degrees

Hãy viết chương trình xác định số lượng số nguyên nằm trong tập $[X, Y]$ và là tổng của đúng K lũy thừa nguyên khác nhau của B .

Ví dụ, với $X = 15$, $Y = 20$, $K = 2$, $B = 2$, có ba số là tổng của đúng hai lũy thừa nguyên khác nhau của 2:

$$17 = 2^4 + 2^0, \quad 18 = 2^4 + 2^1, \quad 20 = 2^4 + 2^2.$$

Input. Dòng đầu chứa hai số nguyên X và Y , cách nhau bởi một dấu cách:

$$1 \leq X \leq Y \leq 2^{31} - 1.$$

Hai dòng tiếp theo chứa K và B :

$$1 \leq K \leq 20, \quad 2 \leq B \leq 10.$$

Output. In ra một số nguyên duy nhất: số lượng số nguyên nằm giữa X và Y và là tổng của đúng K lũy thừa nguyên khác nhau của B .

Ví dụ.

Input:

15 20

2

2

Output:

3

B Đề gốc của ví dụ 2: Sorted bit sequence

Xét biểu diễn 32 bit của mọi số nguyên i từ m đến n , kể cả hai đầu ($m \leq i \leq n$, $mn \geq 0$, $-2^{31} \leq m \leq n \leq 2^{31} - 1$). Số âm được biểu diễn bằng mã bù hai 32 bit: dãy 32 bit của nó cộng nhị phân với biểu diễn 32 bit của số dương tương ứng đúng bằng 2^{32} , tức

$$1\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000_2.$$

Chẳng hạn, biểu diễn 32 bit của 6 là

0000 0000 0000 0000 0000 0000 0000 0110

và biểu diễn 32 bit của -6 là

1111 1111 1111 1111 1111 1111 1111 1010

vì tổng nhị phân của hai dãy này bằng 2^{32} .

Sắp xếp các biểu diễn 32 bit theo thứ tự tăng dần của số bit 1. Nếu hai biểu diễn có cùng số bit 1, sắp xếp chúng theo thứ tự từ điển. Với $m = 0$, $n = 5$, kết quả là bảng đã nêu trong phần ví dụ 2; với $m = -5$, $n = -2$, kết quả cũng như bảng trong phần đó.

Cho m, n, k với

$$1 \leq k \leq \min\{n - m + 1, 2147473547\},$$

hãy tìm số tương ứng với biểu diễn thứ k trong dãy đã sắp xếp.

Input. Dữ liệu gồm nhiều bộ test. Dòng đầu chứa số bộ test, là một số nguyên dương không lớn hơn 1000. Mỗi dòng sau mô tả một bộ test bằng ba số nguyên m, n, k cách nhau bởi dấu cách.

Output. Với mỗi bộ test, in ra trên một dòng số thứ k trong dãy đã sắp xếp.

Ví dụ.

Sample input:

```
2
0 5 3
-5 -2 2
```

Sample output:

```
2
-5
```

C Đề gốc của ví dụ 3: Sequence

Cho dãy gồm mọi số nhị phân K bit:

$$0, 1, \dots, 2^K - 1.$$

Cần chia toàn bộ dãy thành M khúc. Mỗi khúc phải là một dãy con liên tiếp của dãy ban đầu. Gọi S_i ($1 \leq i \leq M$) là tổng số bit 1 trong mọi số của khúc thứ i khi viết ở dạng nhị phân, và gọi S là giá trị lớn nhất trong các S_i . Mục tiêu là tối thiểu hóa S .

Input. Dòng đầu chứa hai số K và M :

$$1 \leq K \leq 100, \quad 1 \leq M \leq 100, \quad M \leq 2^K.$$

Output. In ra một dòng chứa giá trị nhỏ nhất có thể đạt được của S , không có số 0 thừa ở đầu. Kết quả không bảo đảm vừa trong số nguyên 64 bit.

Ví dụ.

Input:

```
3 4
```

Output:

```
4
```

D Đề gốc của ví dụ 4: Tickets

Nghề bán vé khá đơn điệu, vì công việc chỉ là bán vé cho hành khách. Vì vậy, một người bán vé quyết định thay đổi công việc của mình: giờ đây anh ta bán nhiều vé cùng lúc cho mỗi hành khách. Cụ thể, anh ta lần lượt đưa vé cho một hành khách cho đến khi tổng các chữ số trên mọi vé đã đưa không nhỏ hơn một số nguyên k . Sau đó quá trình lặp lại với hành khách tiếp theo.

Ban đầu người bán vé có một dải vé được đánh số liên tiếp từ l đến r , kể cả hai đầu. Cách phát vé này khiến hành khách vui vì được nhận nhiều vé dù chỉ trả tiền cho một vé, nhưng có một bất lợi: vì mỗi hành khách nhận nhiều vé, người bán vé có thể không phục vụ được tất cả hành khách. Hãy tính người bán vé có thể phục vụ bao nhiêu hành khách.

Input. Tập vào chứa ba số nguyên l, r, k :

$$1 \leq l \leq r \leq 10^{18}, \quad 1 \leq k \leq 1000.$$

Output. In ra đúng một số: đáp án của bài toán.

Ví dụ.

Sample input:

40 218 57

Sample output:

29

E Đề gốc của ví dụ 5: Graduated Lexicographical Ordering

Xét các số nguyên từ 1 đến n . Gọi tổng chữ số của một số nguyên là trọng số của nó, ký hiệu trọng số của x là $w(x)$.

Ta sắp xếp các số theo thứ tự gọi là **graduated lexicographical order**, viết tắt là thứ tự grlex. Với hai số nguyên a và b , nếu $w(a) < w(b)$ thì a đứng trước b trong thứ tự grlex. Nếu $w(a) = w(b)$, thì a đứng trước b trong thứ tự grlex khi và chỉ khi biểu diễn thập phân của a nhỏ hơn biểu diễn thập phân của b theo thứ tự từ điển.

Ví dụ:

$$120 <_{\text{grlex}} 4 \quad \text{vì} \quad w(120) = 1 + 2 + 0 = 3 < 4 = w(4),$$

$$555 <_{\text{grlex}} 78 \quad \text{vì} \quad w(555) = 15 = w(78) \text{ và "555" nhỏ hơn "78" theo từ điển,}$$

$$20 <_{\text{grlex}} 200 \quad \text{vì} \quad w(20) = 2 = w(200) \text{ và "20" nhỏ hơn "200" theo từ điển.}$$

Cho n và một số nguyên k , hãy tìm vị trí của số k trong thứ tự grlex của các số nguyên từ 1 đến n , đồng thời tìm số nguyên đứng ở vị trí thứ k trong thứ tự này.

Input. Dữ liệu gồm nhiều dòng, mỗi dòng chứa hai số nguyên n và k :

$$1 \leq k \leq n \leq 10^{18}.$$

Dòng $n = k = 0$ kết thúc dữ liệu.

Output. Với mỗi dòng vào, in ra một dòng. Trước hết in vị trí của số k trong thứ tự grlex của các số nguyên từ 1 đến n , sau đó in số nguyên đứng ở vị trí thứ k trong thứ tự này.

Ví dụ.

Sample Input:

20 10

0 0

Sample Output:

2 14